

F7-LAS

Securing Agentic AI with F7-LAS

F7-LAS Fuller 7 Layer Agentic Security Whitepaper Version 4.0

A governed architecture for policy decisions, approval binding, enforced execution, and verifiable evidence

Anthony L. Fuller

Whitepaper v4.0

September 2026

Publication status

Publication edition

DOI: [10.5281/zenodo.22867553](https://doi.org/10.5281/zenodo.22867553)

Rights and artifact provenance

Copyright © 2026 Anthony L. Fuller. Except where otherwise noted, this whitepaper is licensed under the Creative Commons Attribution 4.0 International license. F7-LAS™ is a trademark of Anthony L. Fuller. The license does not grant trademark rights or imply endorsement.

This v4.0 publication edition is a new revision. It does not overwrite, relabel, or silently replace the v3.0 DOCX or PDF. The v3.0 artifacts remain the historical record associated with their existing archive and DOI.

The technical implementation basis for this edition is repository commit d6a413b878393d98cc2fb2581678f05801767134 and the immutable v4.0.0 repository release. The repository release and this whitepaper publication remain independently versioned artifacts. The repository release does not establish production readiness.

F7-LAS is independent personal work. It is not affiliated with, endorsed by, or representative of Microsoft or any other employer.

How to cite this publication

Fuller, Anthony L. Securing Agentic AI with F7-LAS. F7-LAS Whitepaper v4.0, September 2026. DOI: [10.5281/zenodo.22867553](https://doi.org/10.5281/zenodo.22867553)

Repository basis

Repository: github.com/anthfuller/F7-LAS

Technical baseline commit: d6a413b878393d98cc2fb2581678f05801767134

Abstract

Agentic AI systems can retrieve information, form plans, invoke tools, and affect external state. That capability changes the security problem. A prompt can express intent, but it cannot serve as the final authority for an action. F7-LAS separates security responsibilities across seven layers so that planning, authorization, enforcement, execution, and assurance remain distinct and reviewable.

Version 4.0 reconciles the reference architecture with a verified repository prototype. It defines the policy decision point and policy enforcement point, binds conditional approval to the complete proposed action and executable policy, places tool access behind enforcement, and treats Layer 6 as the execution boundary for permitted actions. It also replaces broad telemetry examples with eight canonical records and distinguishes deterministic evidence integrity from cryptographic provenance.

The repository demonstrates one synthetic, offline, read-only Python and Open Policy Agent workflow. That path is useful evidence for a bounded pattern. It is not a production agent platform, identity system, human approval service, sandbox, network control, observability service, or certification claim.

Executive conclusion

A secure agentic system should treat every proposed action as untrusted data until an external policy decision and enforcement boundary authorize it. Human approval is additional evidence, not a bypass. Only a policy permit that survives independent enforcement may reach scoped execution. Results must then be validated and recorded as correlated evidence.

- Layer 1 prompt controls and Layer 2 grounding narrow behavior but do not grant execution authority.
- Layer 3 produces bounded plans; Layer 4 expresses a complete proposed tool action.
- Layer 5 evaluates policy and enforces the complete decision, approval, scope, obligations, and expiry binding.
- Layer 6 contains permitted execution within a deployment-specific boundary.
- Layer 7 validates outcomes and preserves evidence for monitoring, evaluation, incident response, and governed change.

Contents

- 1 Purpose and assurance boundary
- 2 The agentic security problem
- 3 The F7-LAS model
- 4 Governed execution and approval binding
- 5 Canonical data contracts
- 6 Evidence integrity and deterministic replay
- 7 Layer responsibilities and controls
- 8 Multi-agent and human oversight patterns
- 9 Threat and framework positioning
- 10 Implementation maturity and repository evidence
- 11 Adoption lifecycle
- 12 Engineering review checklist
- 13 Conclusion
- Appendices and references

How to read this edition

Sections 1 through 4 define the assurance boundary, the seven-layer model, and the enforced control loop. Sections 5 and 6 define the canonical evidence model. Sections 7 through 12 translate the architecture into layer responsibilities, deployment questions, maturity evidence, and review criteria.

Architecture statements describe the intended F7-LAS pattern. Repository statements describe only the bounded implementation at the recorded technical baseline. This distinction is repeated where a reader could otherwise mistake a design requirement for an implemented production control.

SECTION 1

Purpose and assurance boundary

Purpose

F7-LAS is a vendor-neutral and protocol-agnostic reference model for designing, reviewing, and governing agentic AI systems. Its purpose is to make security responsibilities explicit before an agent is connected to tools, identities, data, or external systems. The model supports architecture review, threat modeling, control design, implementation planning, and evidence collection.

What this edition establishes

- The seven-layer responsibility model and one cross-cutting software supply-chain domain called Layer S.
- A governed control loop in which approval returns to the policy decision point and cannot authorize execution directly.
- A complete binding between request, proposed action, approval, policy, enforcement, execution result, and audit evidence.
- A canonical record model for deterministic validation and replay of the bounded repository workflow.
- A precise account of what the repository implements, partially implements, and does not implement.

What this edition does not claim

- No claim of production readiness, operational safety, legal compliance, certification, or regulatory approval.
- No claim that prompts, model alignment, or retrieval controls can replace external authorization and enforcement.
- No requirement to record private model reasoning or chain-of-thought. Observable plans, references, proposals, decisions, results, and evaluations are the audit surface.
- No claim that the repository executor supplies an operating-system, container, virtual-machine, account, or network isolation boundary.
- No claim that digest-bound evidence is digitally signed, externally timestamped, or independently attested.

SECTION 2

The agentic security problem

A conventional chatbot usually returns text. An agentic system can select tools, compose arguments, delegate work, and continue through a sequence of actions. Security must therefore govern both information flow and authority to act. The central design question is not whether the model can produce a plausible action. It is whether the system can prove that the exact action was authorized under the applicable policy and executed inside an appropriate boundary.

From output to action

System behavior	Primary risk	Required control response
Generate text	Misleading, unsafe, or sensitive content	Grounding, content policy, validation, user context
Propose an action	Incomplete or manipulated arguments	Typed action contract, target and scope validation
Request approval	Approval detached from what will execute	Complete digest and policy binding with strict validity
Invoke a tool	Unauthorized access or excessive privilege	External PDP decision and PEP enforcement
Execute a change	Blast radius and irreversible effects	Scoped Layer 6 boundary, least privilege, rollback plan
Evaluate results	False success or missing evidence	Result validation, correlated audit records, monitoring

Security invariants

- Generative output is never authority to act.
- A proposed action is immutable security-relevant data before policy evaluation.
- Approval is valid only for its bound action, policy, scope, authority, and open validity interval.
- Only a PDP permit can reach the PEP, and only successful PEP enforcement can reach tool access or execution.
- Execution at or after an approval expiration time is invalid.
- Denied, malformed, expired, or unverifiable inputs fail closed and preserve evidence when admission succeeded.

SECTION 3

The F7-LAS model

F7-LAS separates concerns that are often collapsed into a single prompt or agent loop. The layer number identifies a control responsibility. It does not imply that every runtime event occurs once in a simple numeric sequence. Layer 4 defines a proposed tool action before Layer 5 authorization, while actual Layer 4 tool access occurs only after the permit is enforced and within Layer 6.

Layer	Control concern	Responsibility	Key limitation
1	System prompt	Express role, mission, behavior, refusal, and escalation intent.	Soft policy; the model can misinterpret or ignore it.
2	Grounding and retrieval	Constrain sources, provenance, freshness, classification, and retrieved content.	Retrieved content can be malicious, stale, or unauthorized.
3	Planner and controller	Bound steps, loops, delegation, state transitions, and stop conditions.	A plan is observable intent, not authority.
4	Tools and integrations	Define complete action contracts, targets, arguments, identities, and side effects.	Tool interfaces expand the action surface.
5	External policy engine	Decide and enforce authorization independently of the model.	Must bind the complete action and fail closed.
6	Scoped execution	Contain permitted execution and limit privilege, egress, resources, and blast radius.	Boundary strength depends on the deployed runtime.
7	Monitoring and evaluation	Validate outcomes, correlate evidence, detect misuse, and drive governed improvement.	Telemetry is evidence only when complete and protected.

Layer S software supply chain

Layer S is a supplemental cross-cutting control domain, not an eighth F7-LAS layer. It covers dependency integrity, build and workflow authority, secret scanning, vulnerability review, software bills of materials, release binding, and update governance. These controls affect every runtime layer because compromised code, policy, models, or build inputs can invalidate architectural controls.

SECTION 4

Governed execution and approval binding

The executive control loop below is normative for the v4.0 architecture. Conditional human approval returns to the PDP for reevaluation. The PEP receives only a PDP permit. Tool access and execution occur only after the PEP verifies the complete binding. Deny or block is separate from clarification or referral because the latter requires new input and a new decision.

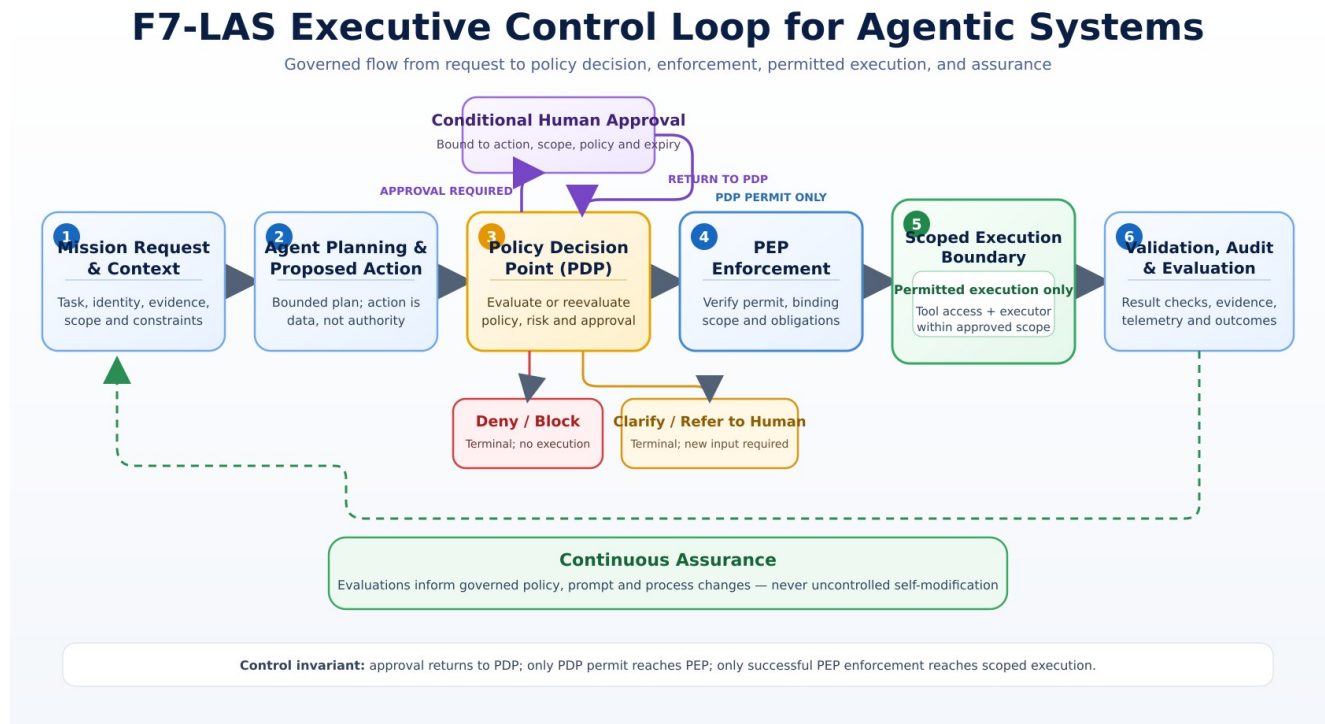


Figure 1 F7-LAS executive control loop for agentic systems

Approval as bound evidence

An approval record is not a general permission slip. It must identify the exact request, proposed action, complete target and arguments, scope or boundary, approving authority, and executable policy bundle. The approval is evaluated by the PDP and rechecked by the PEP and executor. Any mismatch, missing obligation, substitution, or expiration invalidates execution.

Binding element	Required property	Failure response
Request	Reference and digest match the admitted request.	Deny before PEP.
Action	Reference, digest, operation, arguments, and complete target match.	Deny; do not invoke tool.
Scope	Vendor-neutral scope_id or boundary_id matches across records.	Deny as scope mismatch.

Binding element	Required property	Failure response
Authority	Approver identity and authority are present and applicable.	Refer or deny according to policy.
Policy	Policy ID, version, and bundle digest cover metadata and exact Rego bytes.	Fail verification before OPA execution.
Validity	issued_at < expires_at and execution_at < expires_at.	Deny at or after expiration.
Obligations	Every permit obligation is recognized and enforced.	Fail closed on missing or unsupported obligation.

Layer specific runtime sequence

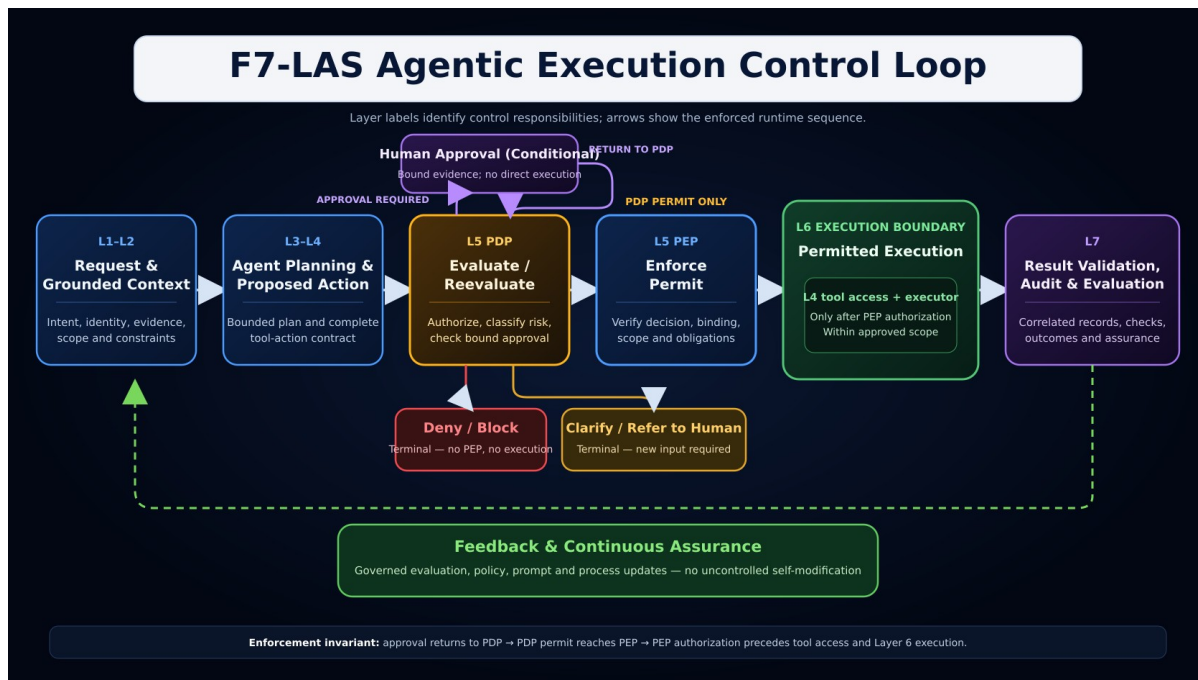


Figure 2 F7-LAS layer specific execution control loop

Decision outcomes

Outcome	Meaning	Next allowed transition
Permit	The complete bound action satisfies policy and required evidence.	PEP verification, then scoped execution.
Deny or block	Policy or enforcement rejects the action.	Terminal for that attempt; no execution.
Clarify	Required context is missing or ambiguous.	Collect new input and submit a new decision.
Refer to human	A person must assess or supply required evidence.	Approval or clarification returns to the PDP.

SECTION 5

Canonical data contracts

The repository defines eight canonical record types at schema version 1.0.0. JSON Schema validates each record independently. A Python cross-record validator checks cardinality, references, ordering, equality constraints, digest binding, and decision-to-execution consistency that separate-record schemas cannot enforce.

Record	Cardinality	Purpose
Request	One per workflow	Mission, actor, scope, constraints, requested outcome
Context	One per workflow	Grounded evidence, source references, environment and boundary
Plan	One per workflow	Bounded observable steps, limits, planner identity
Proposed action	One or more	Complete tool, operation, target, arguments, risk, action digest
Approval	One per action	Status, authority, policy reference, scope, validity and action binding
Policy decision	One per action	Permit or deny, reason, obligations and complete references
Execution result	One per action	Executed or not executed, output, timing, decision binding
Audit event	Zero or more	Correlated events, details and final evidence summary

Shared fields and identifiers

- Every record carries `record_type`, `schema_version`, `workflow_id`, `record_id`, `sequence`, `created_at`, `previous_record_digest`, and `record_digest`.
- Schema version 1.0.0, unique record identifiers, and monotonically increasing sequence values apply across a workflow.
- Timestamps use whole-second RFC 3339 UTC form ending in Z.
- `scope_id` is the required vendor-neutral boundary identifier; `tenant_id` is optional.
- Core objects reject undeclared fields. Record-specific JSON is allowed only where the contract permits it, such as tool arguments, output, or audit details.
- Duplicate JSON keys, NaN, Infinity, secrets, tokens, credentials, and private model reasoning are prohibited.

Deterministic digest domains

Digest	Domain separated input
Record	F7-LAS:record:<record_type>:1.0.0 plus canonical record without record_digest
Action	F7-LAS:action:1.0.0 plus the security-relevant action projection
Output	F7-LAS:output:1.0.0 plus the canonical output object
Policy bundle	F7-LAS:policy-bundle:1.0.0 plus metadata and SHA-256 of exact Rego bytes
Evidence set	F7-LAS:evidence-set:1.0.0 plus the complete canonical evidence document

The implementation uses RFC 8785 JSON Canonicalization Scheme rules. Object member order is immaterial and array order is material. Digest values use the form sha256 followed by a colon and 64 lowercase hexadecimal characters. Record references bind both record_id and record_digest. Action-specific records also bind action_digest.

SECTION 6

Evidence integrity and deterministic replay

What the evidence chain proves

The canonical verifier recomputes every record digest, validates the previous-record chain, checks the complete cross-record relationship set, and confirms that the final audit event accounts for the preceding records in order. The OPA adapter verifies the policy-bundle digest before it executes the Rego policy. Modified policy bytes, substituted records, mismatched references, duplicate keys, and inconsistent decisions fail verification.

What deterministic replay does

Replay reruns the same admitted input through the reviewed synthetic workflow and compares the complete canonical evidence document. This catches nondeterministic or inconsistent changes within the bounded path. Replay performs local OPA evaluation and the synthetic in-process action only; it does not repeat external side effects.

Assurance limits

- The evidence is digest-bound but not digitally signed.
- No trusted external timestamp, transparency log, build attestation, or provenance service is provided.
- A party able to replace both the evidence and the expected digest can create another internally consistent set.
- The replay result applies to the reviewed synthetic action and policy. It does not prove the past state of an external system.
- Output path protections prevent replay from overwriting or hard-linking the input evidence, but they do not provide general filesystem isolation.

SECTION 7

Layer responsibilities and controls

Layer 1 System prompt

State the mission, permitted role, refusal rules, escalation criteria, data handling expectations, and prohibition on treating generated content as authority. Validate prompt structure and protect changes through review. Prompt controls remain soft policy.

Layer 2 Grounding and retrieval

Maintain approved source inventories, ownership, classification, freshness, provenance, and retrieval authorization. Treat retrieved text as untrusted data. Prevent retrieved instructions from changing tool authority or system policy.

Layer 3 Planner and controller

Use observable plans with bounded steps, actions, duration, retries, recursion, and delegation. Record stop conditions and state transitions. Do not require or retain private reasoning traces.

Layer 4 Tools and integrations

Describe each action with a typed contract covering tool, operation, target, arguments, actor, risk, scope, and expected side effects. Separate proposal construction from actual tool access. Validate inputs and outputs independently.

Layer 5 External policy engine

Evaluate authorization outside the model. Bind decisions to the complete action, scope, approval, time, policy bundle, and obligations. Fail closed for malformed, unverifiable, expired, or unsupported inputs. Enforce the permit at a PEP before tool access.

Layer 6 Scoped execution

Execute only after enforcement, using deployment-specific isolation, least privilege, egress restrictions, resource limits, rollback, and lifecycle controls. The repository demonstrates the interface with an in-process executor but does not implement these production boundaries.

Layer 7 Monitoring and evaluation

Correlate request, action, decision, result, and audit evidence. Validate outcomes and side effects. Detect misuse and drift, preserve incident evidence, measure control performance, and route changes through governed review.

Layer S controls

Supply-chain controls should lock dependencies, pin workflow actions and downloaded tools immutably, verify digests, minimize workflow permissions, scan for recognized vulnerabilities and secret patterns, produce an SBOM for the exact release input, and subject updates to review. These

controls reduce known classes of drift and substitution. They do not prove that dependencies, build runners, or releases are non-malicious.

SECTION 8

Multi agent and human oversight patterns

Multi agent systems

A multi-agent design multiplies identities, delegation paths, contexts, and action surfaces. Each agent should have a distinct purpose, allowed tools, scope, and delegation contract. A coordinator may assign work, but it should not transfer authority implicitly. Every action that can affect an external system must still become a complete proposal, pass the applicable PDP and PEP, and execute within a scoped boundary.

Concern	Required evidence
Agent identity	Stable subject identifier, owner, purpose and lifecycle state
Delegation	Caller, recipient, task, scope, expiry, allowed onward delegation
Context transfer	Source, classification, integrity and minimum necessary fields
Action ownership	The proposing and executing identities and complete action digest
Shared tools	Per-agent entitlements, PEP decision and separate execution credentials
Failure containment	Stop conditions, bounded retries, circuit breakers and incident routing

Human oversight

Human involvement can occur before, during, or after an attempted action. Pre-action approval can supply required evidence, but it must be bound and reevaluated. Mid-action intervention requires an explicit interrupt and state model. Post-action review can validate outcomes and tune policy, but it cannot retroactively authorize an action that lacked authority when executed.

- Define which actions require a person and which authority is sufficient.
- Show the exact action, target, arguments, scope, policy, evidence, expected side effects, and validity window to the approver.
- Record approve, deny, clarify, and refer outcomes distinctly.
- Reject zero-length windows and treat the approval as invalid at or after `expires_at`.
- Return the approval evidence to the PDP; never connect an approval button directly to a tool.

SECTION 9

Threat and framework positioning

F7-LAS is an architecture and control-responsibility model. It complements threat catalogs, risk frameworks, security management systems, and application security guidance. A crosswalk can help organize evidence, but it is interpretive. It does not establish conformity, certification, or legal compliance.

Reference	How it complements F7-LAS	Boundary
NIST AI RMF 1.0 and AI 600-1	Risk governance, measurement, management and generative-AI risk considerations.	F7-LAS supplies an agentic control decomposition, not NIST conformity.
CSA MAESTRO	Multi-Agent Environment, Security, Threat, Risk, and Outcome threat modeling for agentic systems.	Threat identification must still be translated into system-specific controls and tests.
OWASP GenAI Security Project	Threat and mitigation guidance for LLM and generative AI applications.	OWASP lists do not replace authorization, containment or evidence architecture.
MITRE ATLAS	Adversary tactics and techniques for AI-enabled systems.	Technique mapping is not proof that a detection or mitigation works.
MITRE ATT&CK	Enterprise adversary behavior relevant to tools, identities, infrastructure and operations.	Use alongside AI-specific threats and deployment context.
ISO IEC 42001	AI management system processes, governance and continual improvement.	Any mapping is interpretive and does not confer certification.

Representative threats and control placement

Threat	Primary layers	Control response
Prompt injection or instruction conflict	1, 2, 5	Separate soft instructions from external authorization; classify retrieved content; fail closed.
Retrieval poisoning	2, 7	Source governance, provenance, ingestion review, content testing and monitoring.
Goal or role drift	1, 3, 7	Stable mission, bounded plan, state validation and behavioral evaluation.
Tool argument manipulation	4, 5	Typed complete action, digest binding, PDP evaluation and PEP revalidation.
Excessive agency	3, 4, 5, 6	Action and loop limits, least privilege, approval tiers and contained execution.
Approval replay or substitution	5, 7	Action, policy, scope, authority and strict-expiry binding; correlated evidence.
Policy substitution	5, S	Policy-bundle digest covering metadata and exact executable bytes.
Agent-to-agent privilege transfer	3, 4, 5	Explicit delegation contract and per-agent tool authorization.
Evidence tampering	7, S	Canonical digests, chain validation, trusted comparison and protected release process.
Dependency or workflow compromise	S	Hash locks, immutable workflow references, verified downloads, audit and SBOM.

SECTION 10

Implementation maturity and repository evidence

The repository at commit `d6a413b878393d98cc2fb2581678f05801767134` is a prototype reference implementation. It includes one synthetic, offline, read-only action that follows the canonical Layers 1 through 7 path through Python and Open Policy Agent. The repository does not make the broader illustrative examples executable or production-ready.

Status	Controls	Meaning in the repository
Implemented	6	Verified for the stated bounded repository behavior and cited evidence.
Partial	29	Some structure, configuration, code or tests exist, with material limitations.
Not implemented	16	No executable control is provided; documentation may describe the intended pattern.
Total	51	46 core Layers 1-7 controls plus five supplemental Layer S controls.

Verified repository capabilities

- One versioned schema defining eight canonical record types with within-record and cross-record validation.
- Duplicate-key rejection and safe output-path checks for the canonical CLI.
- OPA policy evaluation bound to the complete action and the executable policy bundle.
- PEP and executor checks for action, scope, approval, obligations and strict expiration.
- Behavioral scenarios for permit, denial, admission refusal, tampering, failure and recovery.
- Digest verification and deterministic replay for the supported synthetic workflow.
- Hash-locked Python dependencies, immutable workflow references, verified OPA and Gitleaks downloads, vulnerability review, secret scanning and CycloneDX SBOM generation.
- Machine-readable traceability for all 51 catalog controls and a clean-user acceptance gate.

Unimplemented production boundaries

- Production identities, credentials, cloud roles and authorization services.
- Interactive human approval with identity proofing and authority governance.
- Operating-system, container, account, filesystem or network isolation for Layer 6.
- Production tool integrations, external writes, autonomous remediation or rollback.
- Centralized telemetry, detection rules, alerting, incident routing and retention enforcement.
- Digital signatures, trusted timestamps, transparency logs and build or evidence attestations.
- Full behavioral enforcement of every catalog control or illustrative example.

SECTION 11

Adoption lifecycle

Organizations should adopt F7-LAS as a review and engineering method rather than as a checklist whose completion implies safety. Start with an explicit mission and action inventory, then design authorization, execution boundaries, evidence and operational ownership around the real deployment.

1. Inventory agents, identities, data sources, tools, targets, write capabilities, delegation paths and external side effects.
2. Classify actions by impact and define which outcomes are prohibited, permitted, conditional, or require new context.
3. Document the seven-layer architecture and identify every transition where data could be mistaken for authority.
4. Define complete action contracts and external policy inputs before connecting real tools.
5. Implement approval binding, policy evaluation and PEP enforcement before any execution path.
6. Design Layer 6 boundaries using real identity, compute, filesystem, egress, resource and lifecycle controls.
7. Define canonical evidence, validation, telemetry, alerting, incident response, retention and privacy rules.
8. Test positive, negative, boundary, expiry, tampering, replay, failure and recovery scenarios.
9. Map repository or deployment evidence to each control and record limitations honestly.
10. Route policy, prompt, tool, dependency and process changes through governed review and regression testing.

Maturity interpretation

Maturity is deployment-specific. A mature document set does not compensate for an unenforced execution path, and a passing test suite does not establish production assurance outside its tested boundary.

Level	Description	Evidence expected
1 Documented	Responsibilities and boundaries are named.	Architecture, inventory, owners and explicit limitations.
2 Structured	Contracts and review processes are repeatable.	Schemas, policies, action catalog, approval rules and test plan.
3 Enforced	External policy and runtime boundaries control actions.	PDP and PEP records, identities, sandbox and denial evidence.
4 Measured	Outcomes and control performance are monitored.	Telemetry, detections, SLOs, incident evidence and evaluations.
5 Assured	Independent review and governed improvement are routine.	Traceability, release provenance, audits and verified corrective actions.

SECTION 12

Engineering review checklist

Review area	Question
Mission and scope	Are actor, purpose, data, target, boundaries, prohibited outcomes and owners explicit?
Grounding	Are sources authorized, classified, current, attributable and treated as untrusted input?
Planning	Are steps, actions, retries, loops, delegation, duration and stop conditions bounded?
Action contract	Does the proposal include the complete tool, operation, arguments, target, scope, actor, risk and side effects?
Policy decision	Does an external PDP evaluate the complete action and executable policy reference?
Approval	Is approval bound to request, action, authority, policy, scope and a nonzero validity window?
Enforcement	Does a PEP independently verify the permit, obligations, scope and expiry before tool access?
Execution	Does Layer 6 provide appropriate identity, egress, resource, filesystem, lifecycle and blast-radius controls?
Outcome	Are results and side effects independently validated against the permitted action?
Evidence	Are records correlated, canonicalized, protected, retained and available for incident response?
Supply chain	Are dependencies, workflows, downloads, secrets, vulnerabilities, SBOM and releases governed?
Assurance	Do negative tests cover substitution, tampering, expiry boundaries, unsupported obligations and recovery?

Decision rule

If the reviewer cannot identify the exact policy, identity, scope, action, enforcement boundary, execution boundary, expected outcome, and evidence for an external effect, the action should not be enabled. Missing context should produce clarification or human referral. Failed authorization or enforcement should produce denial without execution.

SECTION 13

Conclusion

F7-LAS treats agentic security as an authority and execution problem, not only a content-safety problem. Prompts, grounding and planning shape behavior, but external policy and enforcement determine whether a complete action may proceed. Human approval supplies bound evidence and returns to policy evaluation. Scoped execution follows enforcement, and correlated results support verification and governed improvement.

The v4.0 architecture is aligned with the repository prototype while keeping its limitations visible. The prototype demonstrates canonical contracts, a bounded OPA decision path, approval binding, fail-closed enforcement, evidence integrity, replay and supply-chain checks. Production deployments must still implement real identity, approval, tool, isolation, telemetry and provenance controls and verify them in their own environment.

APPENDIX A

Appendix A Contract validation invariants

Invariant	Validation owner
Exactly one request, context and plan per workflow	Cross-record validator
Exactly one approval, decision and result per proposed action	Cross-record validator
Unique record and plan step identifiers	Schema and cross-record validator
Whole-second UTC timestamps and strict ordering	Schema and cross-record validator
Approval issued before expiration; decision and execution strictly before expiration	Schema, validator, Rego, PEP and executor
Matching request, action, scope, policy and digest references	Cross-record validator, Rego, PEP and executor
Permit required for execution and failed results contain required error evidence	Schema and cross-record validator
Duplicate JSON keys rejected	Canonical loader
Input and output paths are not the same file, symlink target or hard link	CLI preflight

APPENDIX B

Appendix B Repository assurance record

Item	Recorded value
Technical baseline	d6a413b878393d98cc2fb2581678f05801767134
Repository version	F7-LAS 4.0.0 released from annotated tag v4.0.0 at the technical baseline
Immutable tag	https://github.com/anthfuller/F7-LAS/tree/v4.0.0
Canonical contract version	1.0.0
OPA validation version	1.20.2 in the clean-user acceptance environment
Python acceptance version	3.12.14
Automated tests at baseline	119 passed
Golden structural dataset	4 scenarios, 1.00 success rate
Control traceability	51 total: 6 implemented, 29 partial, 16 not implemented
Approved diagram dimensions	1672 by 941 pixels each
Executive diagram SHA-256	23449ac61fc65089d96956d5900916f69ee637960d83a46ff847093e3da59159
Layer diagram SHA-256	9f4400b86796f1f047e51f595f416be5c2c398801a146778833a821277cdd8d6

These values describe the released repository baseline. The repository release does not publish this whitepaper or prove production readiness.

APPENDIX C

Appendix C Glossary

Term	Meaning in this whitepaper
Action digest	Deterministic digest of the security-relevant proposed-action projection.
Approval	Bound evidence from an authorized reviewer; never direct authority to execute.
Boundary	The identity, compute, filesystem, network, resource and lifecycle constraints around execution.
Canonical record	A versioned request-through-audit record with deterministic serialization and digest rules.
Clarify	A terminal outcome for the attempt that requests missing or corrected input.
JCS	JSON Canonicalization Scheme defined by RFC 8785.
Layer S	The supplemental cross-cutting software supply-chain control domain.
Obligation	A condition attached to a permit that enforcement must recognize and satisfy.
PDP	Policy decision point that evaluates the complete authorization request.
PEP	Policy enforcement point that verifies and applies the decision before access.
Policy bundle	Versioned policy metadata plus the exact executable policy bytes covered by one digest.
Proposed action	A complete typed description of a potential tool operation; data, not authority.
Refer to human	A terminal outcome for the attempt that requires human evidence or judgment.
Replay	Deterministic re-execution of the bounded canonical workflow for evidence comparison.

APPENDIX D

References

1. F7-LAS repository technical baseline.
github.com/anthfuller/F7-LAS/tree/d6a413b878393d98cc2fb2581678f05801767134
2. NIST Artificial Intelligence Risk Management Framework AI RMF 1.0. www.nist.gov/itl/ai-risk-management-framework
3. NIST AI 600-1 Generative Artificial Intelligence Profile. www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence
4. Cloud Security Alliance Agentic AI Threat Modeling Framework MAESTRO.
cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro
5. OWASP Top 10 for Large Language Model Applications. owasp.org/projects/top-10-for-large-language-model-applications
6. MITRE ATLAS. atlas.mitre.org/
7. MITRE ATT&CK. attack.mitre.org/
8. Open Policy Agent documentation. www.openpolicyagent.org/docs/latest/
9. RFC 8785 JSON Canonicalization Scheme. www.rfc-editor.org/rfc/rfc8785
10. Creative Commons Attribution 4.0 International. creativecommons.org/licenses/by/4.0/